

Anime.js

Anime.js est une bibliothèque JavaScript légère et flexible pour l'animation. Elle permet de créer des animations complexes avec une syntaxe simple et intuitive. Anime.js prend en charge les animations CSS, les transformations SVG, les propriétés DOM et les objets JavaScript.

Installation via npm

Si vous utilisez npm, vous pouvez installer Anime.js en exécutant la commande suivante :

```
npm install animejs --save
```

Ensuite, vous pouvez importer Anime.js dans votre fichier JavaScript :

```
import { animate } from "animejs";
```

Animer un ou des éléments

Pour animer un ou plusieurs éléments, vous pouvez utiliser la fonction `animate` en spécifiant les éléments cibles et les propriétés à animer. Voici un exemple simple qui anime un élément avec la classe `.box` en le déplaçant de 250 pixels sur l'axe X sur une durée de 2 secondes avec une fonction d'accélération "easeInOutQuad"

Le premier paramètre est le sélecteur CSS des éléments à animer, et le second paramètre est un objet contenant les propriétés d'animation.

```
anime(".box", {  
  translateX: 250,  
  duration: 2000,  
  easing: "easeInOutQuad",  
});
```

Propriétés d'animation courantes

- `duration`: Durée de l'animation en millisecondes.
- `easing`: Fonction d'accélération pour l'animation <https://animejs.com/documentation/easings/built-in-eases#list-of-built-in-easing-functions>.
- `delay`: Délai avant le début de l'animation en millisecondes.
- `loop`: Nombre de fois que l'animation doit se répéter. Utilisez `true` pour une boucle infinie.
- `direction`: Direction de l'animation. Peut être `normal`, `reverse`, ou `alternate`.
- `opacity`: Opacité de l'élément (valeurs entre 0 et 1).
- `autoplay`: Démarrer automatiquement l'animation (valeurs `true` ou `false`).

Propriétés de transformation et CSS courantes

Vous pouvez animer diverses propriétés de transformation et CSS avec Anime.js. Si vous fournissez des tableaux de valeurs, Anime.js interpolera entre ces valeurs pendant l'animation.

<https://animejs.com/documentation/animation/animatable-properties>

- `translateX`, `translateY`, `translateZ`: Déplacement sur les axes X, Y et Z.
- `rotate`, `rotateX`, `rotateY`, `rotateZ`: Rotation autour des axes X, Y et Z.
- `scale`, `scaleX`, `scaleY`: Mise à l'échelle de l'élément.
- `width`, `height`: Largeur et hauteur de l'élément.
- `backgroundColor`: Couleur de fond de l'élément.
- `color`: Couleur du texte de l'élément.
- `borderRadius`: Rayon des bordures de l'élément.

```
anime(".box", {
  backgroundColor: ["#FFF", "#000"],
  borderRadius: ["0px", "50%"],
  duration: 1500,
  easing: "easeInOutQuad",
});
```

Ajout de clés d'animation

Vous pouvez également définir des clés d'animation pour créer des animations plus complexes qui animent plusieurs propriétés à la fois.

```
anime(".box", {
  keyframes: [{ translateX: 250, backgroundColor: "#FFF" }, { translateY: 100,
  backgroundColor: "#000" }, { translateX: 0 }, { translateY: 0 }],
  duration: 4000,
  easing: "easeInOutQuad",
});
```

Décalages entre les éléments animés

Anime.js permet de définir des décalages entre les éléments animés en utilisant la propriété `stagger` en collaboration avec la propriété `delay`. Cela permet de créer des effets d'animation en cascade. Il faut importer `stagger` depuis "animejs" dans le haut de votre fichier JavaScript. Cela va créer un décalage de 100 millisecondes entre le début de l'animation de chaque élément avec la classe `.square`.

```
import { animate, stagger } from "animejs";
```

```
animate(".square", {  
  x: "17rem",  
  delay: stagger(100),  
});
```